

Personal Finance Manager 維護教學

換新電腦、借用公司電腦，或在非自己電腦上維護與部署 web app 的流程

專案	Personal-Finance-Manager
Firebase Project	personal-finance-manager-8e8b4
正式網址	https://personal-finance-manager-8e8b4.web.app
適用日期	2026-04-28

核心原則：在非自己電腦上維護時，盡量只做「暫時環境設定」，不要把公司電腦專用路徑、暫存 config、build output 或登入資料寫進 Git。回到自己電腦時，專案應該仍然可以照常使用。

1. 你需要安裝甚麼

- Git：用來 clone、pull、commit、push 程式碼。
- Node.js 與 npm：用來安裝依賴、開發、build 前端。
- Firebase CLI：用來部署 Hosting、Firestore rules、Cloud Functions。
- Firebase 權限：登入有專案權限的 Google account。
- .env：本機環境變數檔案，不應提交到 Git。

2. 新電腦基本檢查

開 PowerShell 或 Terminal，先檢查工具是否存在：

```
git --version
node --version
npm --version
```

如果未安裝 Node.js，建議用 Node.js LTS。你的 Cloud Functions 設定使用 Node 22，所以新電腦最好安裝 Node 22 LTS。

```
nvm install 22
nvm use 22
node --version
npm --version
```

注意：如果是公司電腦，NVM 可能裝得到但 PATH 接不上，或者 PowerShell 禁止執行 .ps1。這通常是電腦環境限制，不代表 app 壞了。

3. 取得專案

第一次在新電腦工作：

```
git clone <你的-repo-url>
```

```
cd Personal-Finance-Manager
```

如果資料夾已存在，開始工作前先更新：

```
git pull  
git status
```

4. 安裝依賴

根目錄安裝前端依賴：

```
npm install
```

Functions 目錄也要安裝：

```
npm --prefix functions install
```

5. 準備 .env

`.env` 不應放上 Git，所以新電腦要自己建立：

```
copy .env.example .env
```

然後打開 `.env`，填回 Firebase 或 API 相關設定。不要把真實 secret commit 入 Git。

關於 **GEMINI_API_KEY**：目前 Cloud Function 使用 Firebase Secret Manager 的 `GEMINI_API_KEY`，不應硬寫入 repo。部署時 Firebase 會讀取雲端 Secret。

6. 本機檢查

改程式前後，建議跑以下檢查：

```
npm run typecheck  
npm --prefix functions run build  
npm run build
```

三個都成功，代表 TypeScript、Functions 與前端 production build 基本正常。

7. 本機開發

開發時啟動 Vite：

```
npm run dev
```

正常會顯示類似：

```
http://localhost:5173
```

8. Firebase 登入與專案確認

第一次 deploy 前，安裝並登入 Firebase CLI：

```
npm install -g firebase-tools
firebase login
firebase projects:list
```

確認清單內見到：

```
personal-finance-manager-8e8b4
```

你的 repo 內 `.firebaseerc` 已指向這個 project，正常不需要每次手動指定 project。

9. Deploy 指令

完整部署：

```
firebase deploy
```

這會部署：

- Firestore rules
- Cloud Functions
- Firebase Hosting

只部署前端 Hosting：

```
firebase deploy --only hosting
```

只部署 Functions：

```
firebase deploy --only functions
```

只部署 Firestore rules：

```
firebase deploy --only firestore
```

10. 公司電腦常見問題

問題 A：PowerShell 禁止 npm.ps1 / firebase.ps1

如果見到類似：

```
npm.ps1 cannot be loaded because running scripts is disabled
```

不要改專案，用 `.cmd` 版本即可：

```
npm.cmd install
npm.cmd run build
```

```
firebase.cmd deploy
```

問題 B : NVM 裝了但 node / npm 找不到

先找 NVM 的 Node 版本資料夾，常見位置：

```
C:\Users\<你個user>\AppData\Local\nvm\v22.x.x
```

只在目前 PowerShell 視窗暫時補 PATH：

```
$env:Path='C:\Users\<你個user>\AppData\Local\nvm\v22.x.x;C:\Users\<你個user>\AppData\Roaming\npm;' +  
$env:Path
```

再用：

```
npm.cmd run build  
firebase.cmd deploy
```

安全點：這種 `$env:Path` 做法只影響目前 PowerShell 視窗。關掉視窗後就失效，不會寫入 repo，也不會影響你回家後自己的電腦。

11. Deploy 後檢查

部署成功後會看到：

```
Deploy complete!  
Hosting URL: https://personal-finance-manager-8e8b4.web.app
```

再檢查本機有沒有被 deploy 工具改到：

```
git status
```

如果只見到 Firebase cache，例如：

```
.firebase/hosting.ZGlzdA.cache
```

通常不需要 commit。最好不要把公司電腦產生的快取變更帶回 repo。

12. 日常最短流程

```
git pull  
npm install  
npm --prefix functions install  
npm run typecheck  
npm run build  
npm --prefix functions run build  
firebase login  
firebase deploy
```

13. 維護守則

- 不要 commit `.env`、secret、token 或登入資料。
- 不要把公司電腦專用 PATH 寫入 `package.json`。
- 不要為了一台電腦的限制改 Firebase 設定，除非你確定所有電腦都需要。
- 每次 deploy 前先跑 build。
- 每次 deploy 後跑 `git status`，保持 repo 乾淨。
- 如果環境有問題，優先用暫時 PowerShell 變數解決，不要污染專案。